

AI-Powered Test Automation with Playwright MCP

Playwright MCP (Model Context Protocol) combines **Artificial Intelligence (AI)** with **Playwright** to automate browser testing using **plain English prompts**.

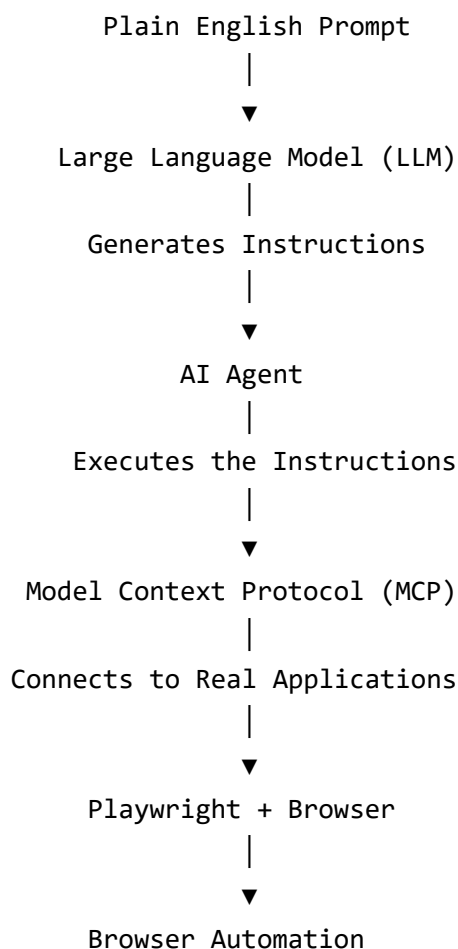
Instead of writing every line of automation code manually, you can describe what you want to test in natural language, and AI helps generate and execute the automation.

Playwright MCP is built on **three important components**:

- **Large Language Model (LLM)** – Understands your request and generates instructions.
- **Agent** – Executes the instructions automatically.
- **Model Context Protocol (MCP)** – Connects AI with real tools such as browsers, APIs, databases, and file systems.

Understanding these three components makes learning Playwright MCP much easier.

Architecture Overview



1. Large Language Model (LLM)

What is an LLM?

A **Large Language Model (LLM)** is an AI model trained on massive amounts of text and code. It understands human language and generates intelligent responses.

You communicate with an LLM using **natural language**, just as you would talk to another person.

Popular LLMs

- ChatGPT
- Claude
- Gemini
- DeepSeek

What Can an LLM Do?

An LLM can:

- Understand plain English instructions
- Read requirements and user stories
- Generate Playwright automation code
- Create test cases
- Explain automation concepts
- Suggest improvements
- Help debug automation scripts

Example

Prompt

Login to the application, search for "Laptop", add the first product to the cart, and verify the cart count.

The LLM understands the request and generates the required Playwright automation code.

What an LLM Cannot Do

Although an LLM is intelligent, it **cannot perform real-world actions** by itself.

It cannot:

- Open a browser
- Click buttons
- Type into input fields

- Execute Playwright tests
- Access your local files
- Interact directly with APIs or databases

It only **generates instructions**.

Simple Understanding

LLM = What to Do

Think of the LLM as the **Brain** or **Idea Generator**.

It decides **what actions should be performed**, but it does not perform them itself.

2. Agent

What is an Agent?

An **Agent** is an intelligent assistant that takes the instructions generated by the LLM and performs real tasks automatically.

Unlike an LLM, an Agent can interact with applications, browsers, and other systems.

What Can an Agent Do?

An Agent can:

- Execute browser automation
- Navigate websites
- Read system alerts
- Call APIs
- Access databases
- Fetch logs
- Restart services
- Complete multi-step workflows
- Perform repetitive tasks automatically

Example

The LLM generates the following instruction:

Open the login page and sign in.

The Agent performs the following steps automatically:

<https://www.pavanonlinetrainings.com>

<https://www.youtube.com/@sdetpavan>

1. Opens the browser
2. Navigates to the website
3. Enters the username
4. Enters the password
5. Clicks the Login button

Key Features

- Requires little or no human intervention
- Converts AI-generated ideas into real actions
- Can execute complex workflows automatically

Simple Understanding

Agent = How to Do It

Think of the Agent as the **Executor** or **Doer**.

3. Model Context Protocol (MCP)

What is MCP?

Model Context Protocol (MCP) is an open communication protocol that enables AI Agents to communicate with external tools in a standardized way.

It acts as a **bridge** or **translator** between AI and real-world applications.

Without MCP, an AI model cannot directly control browsers or access external systems.

MCP Can Connect To

- Web Browsers
- REST APIs
- Databases
- File Systems
- Local Applications
- Development Tools

What Does MCP Do?

MCP performs the following tasks:

1. Receives instructions from the Agent
2. Converts them into a standard protocol
3. Sends them to the appropriate tool
4. Receives the response
5. Returns the result back to the Agent

Browser Actions Through MCP

Using Playwright MCP, the Agent can perform actions such as:

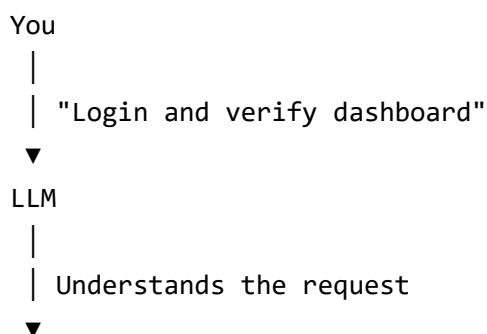
- Open a browser
- Navigate to a webpage
- Click buttons
- Enter text
- Select dropdown options
- Upload files
- Download files
- Take screenshots
- Verify page content
- Execute Playwright commands

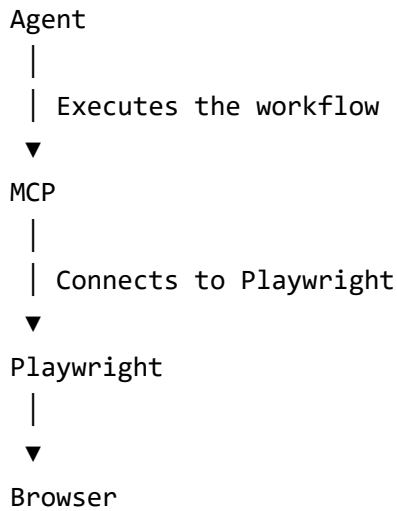
Simple Understanding

MCP = Execution Layer / Translator

It connects AI with real applications and tools.

How LLM, Agent, and MCP Work Together





Roles of Each Component

Component	Role	Simple Meaning
LLM	Generates instructions	What to do
Agent	Executes tasks	How to do it
MCP	Connects AI with external tools	Execution Layer / Translator

Benefits of Playwright MCP

Playwright MCP offers several advantages:

- Write automation using plain English
- Generate Playwright code with AI assistance
- Reduce manual coding
- Create tests faster
- Improve productivity
- Easier for beginners
- Supports AI-assisted maintenance and self-healing capabilities when appropriate
- Simplifies browser automation workflows

Installation and Environment Setup

Follow these three steps to set up Playwright MCP:

1. Enable GitHub Copilot in Visual Studio Code
2. Install Playwright
3. Install the Playwright MCP Server

After completing these steps, you'll be ready to automate browsers using plain English prompts.

Step 1: Enable GitHub Copilot

Prerequisites

- Visual Studio Code
- GitHub Account

Note: GitHub Copilot is integrated into recent versions of Visual Studio Code. You only need to sign in and enable it.

Authenticate GitHub Copilot

1. Open Visual Studio Code.
2. Click the **GitHub** icon.
3. Sign in using your GitHub account.
4. Grant the requested permissions.

After successful authentication:

- GitHub Copilot becomes active.
- The status changes to **Ready**.

Sign Out

To sign out:

1. Click the **Accounts (Profile)** icon in the bottom-left corner of VS Code.
2. Select your GitHub account.
3. Click **Sign Out**.

GitHub Copilot Chat Modes

GitHub Copilot provides three chat modes.

Mode	Purpose
Ask	Ask questions and get explanations
Edit	Modify or improve existing code
Agent	Execute AI-powered automation tasks

Recommendation: Use **Agent Mode** when working with Playwright MCP.

Step 2: Install Playwright

Open the terminal inside your project folder and run:

```
npm init playwright@latest
```

Important: This installs Playwright **only in the current project**. It is **not** a global installation.

Recommended Installation Options

Question	Recommended Answer
Install Playwright	Yes
Language	TypeScript
Test Folder	tests
GitHub Actions	No (Optional)
Install Browsers	Yes

Optional: Playwright Test Extension

You can install the **Playwright Test** extension for Visual Studio Code.

Benefits include:

- Run button
- Debug button
- Test Explorer integration
- Easier debugging
- Beginner-friendly experience
- Less dependency on terminal commands

Step 3: Install the Playwright MCP Server

The Playwright MCP Server can be installed in:

- Visual Studio Code
- Visual Studio Code Insiders

Installation Steps

1. Open the official Playwright MCP GitHub repository. <https://github.com/microsoft/playwright-mcp>
2. Click **Install Server in VS Code**.
3. Follow the installation prompts.

Managing the MCP Server

Open the **Command Palette**:

Ctrl + Shift + P

Search for:

MCP: List Servers

From here, you can:

- Start the server
- Stop the server
- Restart the server
- View the server configuration

MCP Configuration File

The server configuration is stored in:

```
mcp.json
```

You can customize server settings and commands if needed.

Final Setup Checklist

Verify that everything is installed correctly.

- ☒ GitHub Copilot enabled and signed in
- ☒ Playwright installed successfully
- ☒ Browser binaries installed
- ☒ Playwright Test extension installed (Optional)
- ☒ Playwright MCP Server installed and running

Final Result

Your development environment is now ready for **AI-powered Playwright automation**.

You can now:

- Write automation using plain English
- Generate Playwright code with GitHub Copilot
- Execute browser actions through the MCP Server
- Build and run Playwright tests with minimal manual coding

Responsibilities of Each Component

Component	Responsibility
GitHub Copilot (LLM)	Understands prompts and generates automation code
Agent Mode	Executes the requested automation workflow
Playwright MCP Server	Connects AI to Playwright and external tools
Playwright	Automates browser interactions
Browser	Executes the actual UI actions

Complete Workflow

